

Best Practices

Keep these practices in mind for MCP client development:

1. Always use AsyncExitStack

Ensures cleanup even when errors occur:

```
self.exit_stack = AsyncExitStack()
# ... use it for all context managers
await self.exit_stack.aclose() # Cleans everything
```

2. Initialize before operations

Always call `await session.initialize()` after creating the session. This performs the MCP handshake to negotiate protocol version and capabilities with the server.

3. Handle JSON parsing errors

When accepting JSON input from users:

```
try:
    args = json.loads(user_input)
except json.JSONDecodeError:
    print("Invalid JSON format")
    return
```

4. Understand resource templates

- Resource templates use URI patterns with `{parameters}` such as `file://resources/{filename}`
- Discover available templates using `list_resource_templates()`
- Create concrete URIs by substituting actual values for parameters

- One template can dynamically expose many resources without individual definitions

5. Use tool schemas

Check tool schemas from `list_tools()` to know:

- Required vs optional arguments
- Argument types and descriptions
- What the tool does

6. Validate URIs

When reading resources, verify the URI matches an available template pattern to avoid errors.

7. Close cleanly

Always cleanup in a `finally` block to properly close connections and release resources, even when errors occur.